SAFARI ADCS Borealis V1.2 — Test Report

Platform: Raspberry Pi Pico (RP2040)
Framework: Arduino-Mbed (`platform = raspberrypi`, `framework = arduino`)
Toolchain: PlatformIO + Unity (bundled inside `libmbed.a`)
Hardware available: Pico only — no sensors connected (ICM-20948, MS5607, Pt1000, Watchdog absent)
Test environment: `pico_test` (see `platformio.ini`)

Summary

Tier 1 — No extra hardware (bare Pico)

Suite	Tests	Result
`test_pt1000_lut`	5	PASSED
`test_gpio_mux`	3	PASSED
`test_gpio_watchdog`	2	PASSED
`test_gpio_imu`	2	PASSED
`test_adc_pt1000`	2	PASSED
`test_sim_sensor_manager`	5	PASSED
Subtotal	**19**	**19 / 19 PASSED**

Tier 2 — Wire simulation (bare wire between Pico header pins)

Suite	Sensor	Wires required	Tests	Result
`test_pt1000_sim_gpio`	Pt1000 ADC	pin 20 → pin 31	4	not yet run
`test_sim_spi_altimeter`	MS5607 SPI	pin 15 → pin 11	4	not yet run
`test_sim_i2c_imu`	ICM-20948 I2C1	pin 20 → pin 9	4	not yet run
`test_sim_i2c_watchdog`	Watchdog I2C0+CLK	pin 20→pin 6, pin 25→pin 21	5	not yet run
Subtotal			**17**	**pending**

Environment Configuration

```
[env:pico_test]
platform        = raspberrypi
board           = pico
framework       = arduino
lib_deps =
    adafruit/Adafruit ICM20X@^2.0.5
```

```
    adafruit/Adafruit BusIO
    adafruit/Adafruit Unified Sensor
lib_extra_dirs  = Libnexamples
test_build_src  = no
monitor_speed   = 115200
build_flags     = -Wl,--allow-multiple-definition
```

Two flags are mandatory and non-obvious:

- **`test_build_src = no`** — prevents `src/main.cpp`'s `setup()`/`loop()` from being compiled during test builds, which would create duplicate entry-point symbols with the Unity runner.
- **`-Wl,--allow-multiple-definition`** — the Arduino-Mbed framework bundles `unity.o` inside `libmbed.a`; PlatformIO also links its own `libUnity.a`. Without this flag, the linker aborts with `multiple definition of 'UnityDefaultTestRun'`. The flag keeps the first definition and discards the duplicate.

Test Suites

1. `test/test_pt1000_lut/` — LUT Binary-Search Logic

Validates the Pt1000 calibration lookup table without any GPIO or peripheral use.
The 200-entry `LUT_V[]` array and `lutLookup()` function are copied locally from `pt1000.cpp` so the test is fully self-contained.

Calibration constants:
Range −60 °C → +80 °C, 200 entries, step ≈ 0.7035 °C/entry.

Test	What it checks
`test_lut_lower_bound`	`lutLookup(LUT_V[0])` returns −60.0 °C ± 0.01
`test_lut_upper_bound`	`lutLookup(LUT_V[199])` returns index-198 temperature (floor semantics)
`test_lut_midpoint`	`lutLookup(LUT_V[99])` returns exact mid-table temperature
`test_lut_monotone`	10 sampled adjacent pairs: `T(v_i) ≤ T(v_{i+1})` — no inversion
`test_lut_step_size`	`T(LUT_V[50]) − T(LUT_V[49])` equals `T_CAL_STEP` ± 0.01

Result: 5 / 5 PASSED

---

2. `test/test_gpio_mux/` — MUX Address and Power Pins

Verifies GPIO output drive and readback for the 8:1 analog MUX (CD4051 or equivalent) and the Pt1000 power switch. No sensor hardware required.

Pins under test: GPIO0 (A0), GPIO1 (A1), GPIO2 (A2), GPIO3 (PT1000_SW)

Test	What it checks
`test_mux_pins_toggle`	Each of GPIO0–2: `digitalWrite(HIGH)` → `digitalRead() == HIGH`, then LOW/LOW
`test_pt1000_switch_toggle`	GPIO3: HIGH/LOW drive and readback
`test_channel_select_all`	Encodes channels 0–7 as A2:A1:A0 bit pattern; reads back all 3 pins per channel

Result: 3 / 3 PASSED

---

3. `test/test_gpio_watchdog/` — Watchdog CLK Output

Verifies the Watchdog clock output pin. The I2C0 bus scan (GPIO4/5) is **intentionally excluded** — see Known Limitations below.

Pin under test: GPIO19 (WD_CLK)

Test	What it checks
`test_clk_toggle`	`digitalWrite(HIGH)` → readback HIGH, then LOW → readback LOW
`test_clk_pulse`	Full 50% duty-cycle pulse at `WD_CLK_HALF_US` timing; asserts state at each half-cycle

Result: 2 / 2 PASSED

---

4. `test/test_gpio_imu/` — IMU Address Pin and I2C1 Bus

Verifies the AD0 pin that locks the ICM-20948 I2C address and probes I2C1 (GPIO6/7).
Uses `static TwoWire _wire1(PIN_IMU_SDA, PIN_IMU_SCL)` — `Wire1` is not exported by the Arduino-Mbed variant for the Pico, so the custom instance is mandatory (mirrors `imu.cpp`).

Pins under test: GPIO12 (IMU_AD0), GPIO6/7 (I2C1 SDA/SCL)

Test	What it checks
`test_ad0_low`	GPIO12 driven LOW, `digitalRead()` confirms LOW — locks address to 0x68
`test_i2c1_scan`	Probes 0x68 on I2C1; **non-fatal** — prints result to Serial, no assertion

> **Note:** `test_ad0_high` (address 0x69) was removed — AD0 HIGH is never used in the firmware. The ICM-20948 address is permanently locked to 0x68.

Result: 2 / 2 PASSED

---

5. `test/test_adc_pt1000/` — ADC GPIO26 and MUX Switching

Verifies that the ADC peripheral and MUX switching work correctly. A floating pin still returns a value in [0, 1023] — sensor connection is not required.

Pins under test: GPIO26 (ADC), GPIO0–2 (MUX), GPIO3 (SW)

Test	What it checks
`test_adc_channel0_range`	Powers the circuit, reads channel 0, powers off; result ∈ [0, 1023]
`test_adc_all_channels`	Loops channels 0–7: selects via MUX, reads ADC; each result ∈ [0, 1023]

Result: 2 / 2 PASSED

---

6. `test/test_sim_sensor_manager/` — SensorManager Simulation

Tests `SensorManager` orchestration and `g_adcs` shared data bus using stub modules.
No peripheral code is called — stubs inject hardcoded data into `g_adcs` as real modules would.
Includes `../src/sensors.cpp` directly (required because `test_build_src = no`).

Stub modules: `FakeImuModule`, `FakeAltimeterModule`, `FakePt1000Module`, `FakeWatchdogModule`, `AbsentModule` (always fails `init()`).

Test	Scenario
`test_sim_all_modules_present`	4 stubs init successfully; `readAll()` populates `g_adcs` without crash
`test_sim_imu_data_values`	Verifies `g_adcs.imu.az ≈ 9.81`, `mx ≈ 25.0`, `mz ≈ -42.0`, `valid == true`
`test_sim_pt1000_all_channels`	`active_count == 8`; all 8 `valid[i] == true`; `temps[i] ≈ 20 + i` °C
`test_sim_absent_module_skipped`	`AbsentModule` + `FakeImuModule`: absent init fails, IMU still reads; no crash
`test_sim_partial_failure`	2 present + 2 absent: `readAll()` skips absent modules silently

`setUp()` resets `g_adcs = {}` before each test to prevent state leaking between cases.

Result: 5 / 5 PASSED

Tier 2 — Wire Simulation Suites

These suites require one or two bare wires between Pico header pins.
They test the full acquisition chain of each sensor interface with no actual sensor present.

---

7. `test/test_pt1000_sim_gpio/` — Pt1000 Full Chain (ADC)

Wire: Pico **pin 20** (GP15 / stimulus) → Pico **pin 31** (GP26 / ADC0)

GP15 drives 3.3 V (HIGH) or 0 V (LOW) directly onto the ADC input, simulating the Pt1000 voltage divider output at the two extremes of the calibration range.

Test	Stimulus	Expected
`test_sim_wire_connected`	HIGH then LOW on GP15	delta ≥ 800 LSB between the two reads
`test_sim_high_stimulus`	GP15 HIGH (3.3 V)	raw > 900 and LUT → T > 70 °C
`test_sim_low_stimulus`	GP15 LOW (0 V)	raw < 50 and LUT → T < −50 °C
`test_sim_adc_stability`	GP15 HIGH, 2 reads	|r1 − r2| ≤ 5 LSB

---

8. `test/test_sim_spi_altimeter/` — MS5607 Altimeter SPI Bus

Wire: Pico **pin 15** (GP11 / MOSI) → Pico **pin 11** (GP8 / MISO)

MOSI is looped back to MISO: every byte clocked out appears immediately on the input. CS (GP9), SCK (GP10) and MOSI (GP11) are driven by the SPI1 peripheral.

Test	What it checks
`test_sim_spi_wire_connected`	tx=0x00 → rx=0x00; if MISO floats it returns 0xFF
`test_sim_spi_loopback_byte`	tx=0xAA → rx=0xAA
`test_sim_spi_loopback_pattern`	6-byte burst `[0x00, 0xFF, 0x5A, 0xA5, 0x01, 0xFE]` each echoed back
`test_sim_spi_cs_timing`	CS HIGH at idle → LOW on select → HIGH after deselect

---

9. `test/test_sim_i2c_imu/` — ICM-20948 IMU I2C1 SDA Line

Wire: Pico **pin 20** (GP15 / stimulus) → Pico **pin 9** (GP6 / I2C1 SDA)

`Wire.begin()` is NOT called — GP6 is used as plain `INPUT`.
GP15 drives the SDA line as an external agent would (sensor ACK, data bits).

Test	Stimulus	Expected
`test_sim_sda_wire_connected`	HIGH then LOW	GP6 follows: 1 then 0
`test_sim_sda_driven_low`	GP15 LOW	GP6 reads LOW
`test_sim_sda_driven_high`	GP15 HIGH	GP6 reads HIGH
`test_sim_sda_toggle`	Alternate 5×	GP6 matches each level

> SCL (GP7 / pin 10) can be tested identically with a second wire to any free output pin.

---

10. `test/test_sim_i2c_watchdog/` — Watchdog I2C0 SDA + CLK

Wire A (SDA): Pico **pin 20** (GP15) → Pico **pin 6** (GP4 / I2C0 SDA)
Wire B (CLK): Pico **pin 25** (GP19 / WD_CLK) → Pico **pin 21** (GP16 / monitor)

`Wire.begin()` (I2C0) is NEVER called — avoids the Mbed blocking bug (see Known Limitations). SDA tests use Wire A only. CLK external-readback tests use Wire B only.

Test	Wire	What it checks
`test_sim_sda_wire_connected`	A	HIGH/LOW delta on GP4 confirms wire present
`test_sim_sda_driven_low`	A	GP4 follows GP15 LOW
`test_sim_sda_driven_high`	A	GP4 follows GP15 HIGH
`test_sim_clk_external_high`	B	GP16 reads HIGH while GP19 is HIGH
`test_sim_clk_pulse_external`	B	GP16 tracks each half-cycle of a CLK pulse

How to Run

```
# Tier 1 — bare Pico, no wires
pio test -e pico_test -f test_pt1000_lut
pio test -e pico_test -f test_gpio_mux
pio test -e pico_test -f test_gpio_watchdog
pio test -e pico_test -f test_gpio_imu
pio test -e pico_test -f test_adc_pt1000
pio test -e pico_test -f test_sim_sensor_manager

# Tier 2 — wire simulations (add wires before each run)
pio test -e pico_test -f test_pt1000_sim_gpio      # wire: pin20 → pin31
pio test -e pico_test -f test_sim_spi_altimeter    # wire: pin15 → pin11
pio test -e pico_test -f test_sim_i2c_imu          # wire: pin20 → pin9
pio test -e pico_test -f test_sim_i2c_watchdog     # wires: pin20→pin6,
pin25→pin21

# All suites at once (Tier 1 only — remove wires or run Tier 2 separately)
pio test -e pico_test

# Serial monitor
pio device monitor --baud 115200
```

Known Limitations

I2C0 scan excluded from `test_gpio_watchdog`

The Mbed I2C driver used by the default `Wire` object has no timeout on `Wire.endTransmission()`. When no device acknowledges on I2C0 (GPIO4/5), the call **blocks indefinitely** — observed hang of 20+ minutes in practice. The I2C0 bus scan was therefore removed from `test_gpio_watchdog`. CLK GPIO tests are sufficient to verify the Watchdog pin without triggering this bug.

I2C1 (`_wire1` on GPIO6/7) does not exhibit this problem: `endTransmission()` returned normally in `test_i2c1_scan` even with no sensor present.

I2C scans are non-fatal

`test_i2c1_scan` probes address 0x68 and prints the result to Serial but makes no assertion. The test passes whether or not the ICM-20948 is physically connected. This is intentional — the suite is designed to run on bare hardware.

Issues Resolved During Development

Issue	Root cause	Fix
`'ADC_VREF' redeclared`	RP2040 SDK defines `PinName ADC_VREF` in `PinNames.h`	Renamed to `PT1000_VREF` in `pt1000.cpp`
`Wire.setSDA()/setSCL()` missing	Not available on `MbedI2C` class	Removed calls; `Wire` uses GPIO4/5 by default
`Wire1` undeclared	Not exported by Arduino-Mbed Pico variant	Created `static TwoWire _wire1(6, 7)` manually
`TEST_ASSERT_GREATER_OR_EQUAL` missing	Not available in all Unity versions bundled by Mbed	Replaced with `TEST_ASSERT_TRUE_MESSAGE(raw >= 0, ...)`
`TEST_MESSAGE()` missing	Not in bundled Unity	Replaced with `Serial.println()`
`WATCHDOG_I2C_ADDR`/`WD_CLK_HALF_US` undeclared	Missing include	Added `#include "watchdog.h"`
Duplicate `setup()`/`loop()` link error	`src/main.cpp` compiled during test builds	Added `test_build_src = no`
Multiple Unity symbol definitions	`libmbed.a` bundles `unity.o`; PlatformIO adds `libUnity.a`	Added `-Wl,--allow-multiple-definition`
`test_gpio_watchdog` hung 20 min	`Wire.endTransmission()` blocks on NACK (Mbed, no timeout)	Removed I2C0 scan entirely from that suite
Cascading upload failures on last 2 suites	Pico USB lost after 20-min hang; serial port `PermissionError`	Fixed by removing the blocking test